

UNIVERSITY OF GHANA
COLLEGE OF BASIC AND APPLIED SCIENCE
DEPARTMENT OF COMPUTER SCIENCE



DCIT 204/308: DATA STRUCTURES AND ALGORITHMS I & II
JOINT SEMESTER PROJECT, TECHNICAL REPORT

ECG DUMSOR RESPONSE OPTIMIZER
FAULT DISPATCH AND REPAIR CREW ROUTING FOR THE ECG
ACCRA/LEGON DISTRIBUTION ZONE

GROUP 15: CODEBILITY V2.0

AUGUST 2026

TEAM MEMBERS

ROLAND APPIAH (22197127)

JENNIFER BANIBENSU (22013023)

KINGSLEY ADDO (22241382)

DENNIS KWAKU DAPAAH ADOMAKO (22238484)

PATRICIA GYAN (22141938)

ANGEL FRANCISCA ECHESI (22398675)

DANIEL KWADWO TAKYI (22390064)

ANASTHASIA KODUAH TWENEBOAH (22311176)

HAFISAH IBRAHIM (22381877)

JOEL KISSIEDU AMISSAH (22368505)

SHADRACH ADDOQUAYE ADDO (22396810)

MICHELLE NANA ABENA ASANTEWAA SARFO (22396802)

SAMPSON MENUM LANDOKIDOW (22300655)

SAMUEL PETER PETER (22410937)

LOCAL GHANA CONTEXT: UNIVERSITY CAMPUS SERVICE HUB

Selected organisation and problem: Electricity Company of Ghana (ECG) power outage fault dispatch and repair crew routing across the University of Ghana, Legon campus and the surrounding Accra districts (Okponglo, Madina, Adenta)

Table of Contents

1. PROBLEM STATEMENT, ASSUMPTIONS, INPUT TO OUTPUT DEFINITIONS AND SYSTEM BOUNDARIES.....	5
1.1 THE PROBLEM.....	5
1.2 ASSUMPTIONS.....	5
1.3 INPUT AND OUTPUT DEFINITIONS FOR THE FIVE MAIN OPERATIONS.....	5
Operation 1: Dispatch the next fault.....	5
Operation 2: Shortest route between two locations.....	5
Operation 3: Reachability from a dispatch point.....	6
Operation 4: Budget or capacity constrained selection.....	6
Operation 5: Minimum cost network connection.....	6
1.4 System boundaries.....	6
2. DATASET DESCRIPTION, DATA DICTIONARY AND DATABASE SCHEMA.....	7
2.1 WHY THIS DATASET.....	7
2.2 TABLE BY TABLE DESCRIPTION.....	7
locations.....	7
roads.....	7
resources.....	7
service_requests.....	7
algorithm_runs.....	7
2.3 DATA DICTIONARY.....	7
2.4 DATABASE SCHEMA.....	9
2.5 TEAM PARAMETERS DERIVED FROM A STUDENT INDEX NUMBER.....	9
3. SYSTEM ARCHITECTURE AND MODULE DESIGN.....	11
3.1 LAYERS.....	11
3.2 MODULE COVERAGE.....	11
3.3 DATA STRUCTURE TO APPLICATION MAPPING.....	12
4. DATA STRUCTURE IMPLEMENTATION.....	13
4.1 DYNAMIC ARRAY.....	13
4.2 LINKED LIST, STACK, AND QUEUES.....	13
4.3 PRIORITY QUEUE (BINARY MIN HEAP).....	14
4.4 TREES: BST, RED BLACK TREE, AND B TREE.....	14
4.5 HASHING: HASHTABLE, CUSTOMMAP, CUSTOMSET.....	15
4.6 DISJOINT SET AND GRAPH.....	15
5. ALGORITHM IMPLEMENTATION.....	16
5.1 PSEUDOCODE: PRIORITY DISPATCH.....	16

5.2 PSEUDOCODE: DIJKSTRA SHORTEST ROUTE	16
5.3 PSEUDOCODE: KRUSKAL MINIMUM FEEDER NETWORK	17
5.5 PSEUDOCODE: STACK-BACKED UNDO LOG	18
5.6 GREEDY ALGORITHM: CREW SHIFT SCHEDULING.....	18
5.7 DYNAMIC PROGRAMMING: 0/1 KNAPSACK FOR BUDGET SELECTION.....	18
6. CORRECTNESS EVIDENCE	19
6.1 AUTOMATED TESTS	19
6.2 TRACE TABLES.....	20
6.3 PROOF SKETCH 1: LOOP INVARIANT FOR SELECTION SORT	21
6.4 PROOF SKETCH 2: DIJKSTRA CORRECTNESS BY INDUCTION.....	21
6.5 PROOF SKETCH 3: GREEDY EXCHANGE ARGUMENT FOR ACTIVITY SELECTION.....	21
6.6 COUNTEREXAMPLE 1: GREEDY COIN CHANGE FAILS.....	22
6.7 COUNTEREXAMPLE 2: BINARY SEARCH NEEDS A SORTED ARRAY	22
7. PERFORMANCE ANALYSIS	23
7.1 METHOD	23
7.2 SEARCH COMPARISON	23
7.3 SORTING COMPARISON.....	24
7.4 HASH TABLE LOAD FACTOR AND COLLISIONS	24
7.5 HEAP PRIORITY DISPATCH	25
7.6 BST VERSUS RED BLACK TREE.....	26
7.7 GRAPH ALGORITHMS	27
8. DATABASE INTEGRATION EVIDENCE	29
8.1 FULL SCHEMA	29
8.2 SAMPLE SEED RECORDS	29
8.3 RUN LOGS, ALGORITHM_RUNS TABLE ACCESS	29
8.4 SCREENSHOTS.....	30
8.5 MACHINE SPECIFICATION FOR THE PERFORMANCE RESULTS	33
9. RESPONSIBLE ALGORITHM SELECTION.....	34
9.1 SEARCH.....	34
9.2 SORTING	34
9.3 GRAPH ALGORITHMS	34
9.4 GREEDY VERSUS DYNAMIC PROGRAMMING	34
10. INDIVIDUAL CONTRIBUTION STATEMENT AND ORAL DEFENSE PREPARATION NOTES	35
11. REFERENCES AND APPENDICES	36
11.1 REFERENCES.....	36

11.2 APPENDIX A: AI ASSISTANCE ACKNOWLEDGEMENT	36
11.3 APPENDIX B: QUEUE STRATEGY AND INDEXING DEMONSTRATIONS.....	36
11.4 APPENDIX C: DEVELOPMENT LOG SUMMARY	37
11.5 APPENDIX D: MACHINE SPECIFICATION.....	37

1. PROBLEM STATEMENT, ASSUMPTIONS, INPUT TO OUTPUT DEFINITIONS AND SYSTEM BOUNDARIES

1.1 THE PROBLEM

Ghana's power utility, ECG, deals with frequent outages, known locally as "dumsor" (Twi for "off, on"). When a fault happens, someone must decide which fault gets fixed first, which repair crew goes, and what the fastest route to the fault is. Doing this by hand does not scale once there are dozens of crews and hundreds of outages reports a week.

This project builds a small software system that does three things for the ECG Accra/Legon distribution zone:

- Decide which outage report should be handled next, using FIFO order, urgency order, or a mix of both.
- Work out the fastest route from a repair base to the fault location, and which other locations are reachable from a given point.
- Pick the best subset of jobs or resources to work on when there is a limited budget, limited vehicles, or limited crew time.

Everything is backed by data structures and algorithms the team wrote from scratch, not Java's built-in collections, and the data is stored in a real PostgreSQL database rather than just kept in memory.

1.2 ASSUMPTIONS

- All road and feeder distances are treated as fixed numbers for a given run of the program. Real-time traffic changes are out of scope.
- Urgency is entered by whoever logs the fault, on a 1 to 5 scale (5 is a critical outage such as a hospital or a major substation, 1 is routine maintenance). The system does not try to guess urgency automatically.
- A repair crew can only be assigned to one active job at a time, up to its stated capacity.
- Edge weights (distance, travel time) are never negative, which is required for Dijkstra's algorithm to give correct answers.
- The dataset is a realistic simulation built from real Legon and Accra place names and ECG's actual fault categories, not scraped from a live ECG system.

1.3 INPUT AND OUTPUT DEFINITIONS FOR THE FIVE MAIN OPERATIONS

These are the operations the brief asks for pseudocode and flowcharts on. Full pseudocode is in Section 5.

Operation 1: Dispatch the next fault

- **Input:** the current queue or heap of open fault records, each with an urgency level.
- **Output:** the single fault record to work on next, and the crew assigned to it.
- **Precondition:** at least one fault is open, and at least one crew is available.
- **Edge case:** no crew is available for the required crew type; mark the fault as pending and log it.

Operation 2: Shortest route between two locations

- **Input:** a source location and a destination location, plus the road network graph.
- **Output:** the shortest total distance and the ordered list of locations on that route.
- **Precondition:** all edge weights are non-negative.

- **Edge case:** no path exists (disconnected graph), the system reports that the destination is unreachable rather than crashing.

Operation 3: Reachability from a dispatch point

- **Input:** a starting location and the graph.
- **Output:** the set of every location reachable from that point, and the order in which BFS or DFS visited them.

Operation 4: Budget or capacity constrained selection

- **Input:** a list of candidate jobs or items, each with a cost (weight) and a benefit (value), plus a capacity limit.
- **Output:** the subset of jobs that fits the capacity and gives the highest total benefit, using the 0/1 knapsack dynamic programming approach.

Operation 5: Minimum cost network connection

- **Input:** the set of locations and candidate feeder lines with their costs.
- **Output:** the minimum set of feeder lines needed to connect every location, using Prim's or Kruskal's algorithm.

1.4 System boundaries

In scope: custom data structures, custom algorithms, a PostgreSQL database with CSV fallback, a JavaFX desktop interface for demonstration, and an empirical performance study.

Out of scope: a public-facing web app, live GPS tracking of crews, SMS notifications to customers, and any machine learning-based urgency prediction.

2. DATASET DESCRIPTION, DATA DICTIONARY AND DATABASE SCHEMA

2.1 WHY THIS DATASET

The dataset models ECG's Accra/Legon distribution zone, which supplies power to the University of Ghana campus, Legon residential estates, East Legon, Adenta, Madina, Haatso, and Achimota. Coordinates are approximate decimal degree positions inside the Accra metropolitan area (roughly latitude 5.55 to 5.70 North, longitude minus 0.28 to minus 0.15 West). This keeps the dataset grounded in a real, recognisable place rather than a generic made-up city.

2.2 TABLE BY TABLE DESCRIPTION

locations

50 ECG substations, transformer stations, and customer service points across Legon, East Legon, Adenta, Madina, Haatso, and Achimota.

roads

100 road and underground feeder connections between locations. The weight represents an estimated travel time in minutes for a repair crew vehicle, adjusted by a road condition penalty.

resources

30 repair crews and response units, split into four types: Substation Crew, Overhead Line Crew, Underground Cable Crew, and Emergency Response.

service_requests

300 simulated outage and fault reports across the zone, each with an urgency level from 1 (routine) to 5 (critical, for example a hospital or major substation).

algorithm_runs

Recorded timing and memory measurements from the performance experiments described in Section 7, currently 540 recorded runs.

2.3 DATA DICTIONARY

Table 1. data dictionary of the various csv files

Table	Field	Type	Meaning
locations	location_id	String	Unique ID for the location, primary key
locations	name	String	Human readable name, for example UG Legon Main Substation
locations	area	String	District or neighbourhood, for example Legon, East Legon, Madina
locations	location_type	String	Substation, Transformer Station, Customer Service, and so on

locations	x_coord / y_coord	Float	Approximate latitude and longitude
roads	road_id	String	Unique ID for the road or feeder edge
roads	from_location_id / to_location_id	String	The two locations this edge connects
roads	distance_km	Float	Distance in kilometres
roads	travel_time_min	Integer	Estimated travel time in minutes
roads	condition_weight	Float	Road condition penalty used to adjust the effective edge weight
resources	resource_id	String	Unique ID for the crew or vehicle
resources	resource_type	String	Cable truck, Transformer crew, Meter team, and so on
resources	home_location_id	String	The location this resource is based at
resources	capacity	Integer	How many concurrent jobs this resource can take
resources	availability_status	String	AVAILABLE, BUSY, or OFFLINE
service_requests	request_id	String	Unique ID for the fault report
service_requests	source / destination location id	String	Where the report came from and where the crew must go
service_requests	category	String	Transformer Failure, Meter Fault, Cable Burst, Fallen Conductor
service_requests	urgency	Integer	1 (routine) to 5 (critical)
service_requests	time_submitted / deadline	Timestamp	When the report came in and when it should be resolved by
service_requests	status	String	OPEN or NEW, ASSIGNED, or RESOLVED or COMPLETED
algorithm_runs	algorithm_name	String	Which algorithm this row measured
algorithm_runs	input_size	Integer	Size of the input used for this run
algorithm_runs	time_ns / memory_kb	Long / Integer	Measured runtime and memory

2.4 DATABASE SCHEMA

The system uses PostgreSQL, hosted on Neon.tech, with six tables: locations, roads, service_requests, resources, algorithm_runs, and audit_events (for the stack-based undo log). The full schema lives in database/schema.sql. Key constraints:

- Every road references two existing locations and cannot be a self loop.
- Urgency is constrained to the range 1 to 5.
- A service request's deadline must be on or after its submission time.
- Resource availability is constrained to AVAILABLE, BUSY, or OFFLINE.

locations(location_id PK, name, area, location_type, x_coord, y_coord)
roads(road_id PK, from_location_id FK, to_location_id FK, distance_km, travel_time_min, condition_weight)
service_requests(request_id PK, source_location_id FK, destination_location_id FK, category, urgency, time_submitted, deadline, status)
resources(resource_id PK, resource_type, home_location_id FK, capacity, availability_status)
algorithm_runs(run_id PK, algorithm_name, input_size, time_ns, memory_kb, run_number, date_run)
audit_events(event_id PK, event_type, related_entity_type, related_entity_id, description, event_timestamp)

2.5 TEAM PARAMETERS DERIVED FROM A STUDENT INDEX NUMBER

The brief requires at least three algorithm parameters to be derived from team member index numbers. All four below come from Michelle Nana Abena Asantewaa Sarfo's index number, 22396802, and are confirmed present in src/main/java/com/g15/dsa/database/TeamParameters.java:

Table 2. Team Parameter

Parameter	Derivation	Value
Urgency weight	$1.0 + (\text{digit sum of } 22396802 \bmod 5) \times 0.2$. Digit sum is $2+2+3+9+6+8+0+2 = 32$. $32 \bmod 5 = 2$. $1.0 + 2 \times 0.2 = 1.4$	1.4
Road penalty	$1.0 + (\text{last two digits mod } 10) \times 0.1$. Last two digits are 02. $2 \bmod 10 = 2$. $1.0 + 2 \times 0.1 = 1.2$	1.2
Hash capacity	Next prime number after $100 + (22396802 \bmod 50)$. $22396802 \bmod 50 = 2$, so $100 + 2 = 102$, and the next prime is 103	103
Hash seed	Last four digits of the index number	6802

Where each parameter is confirmed to be wired in:

- Hash capacity (103) and hash seed (6802): confirmed in HashTable.java, used as the default table size and in the index spreading formula. Confirmed by *HashTableTest.testDefaultCapacity_matchesTeamParametersPrime* and *TeamParametersTest.hashTableUsesTeamParametersByDefault*, both of which assert the table's default capacity equals 103.

- Urgency weight (1.4): confirmed in *Fault.java* (*getWeightedUrgency* multiplies urgency by *URGENCY_WEIGHT*) and in *DispatchController.calculateDispatchScore*. Confirmed by *TeamParametersTest.faultModelUsesUrgencyWeight* and *dispatchScoringIncorporatesParameters*.
- Road penalty (1.2): confirmed in *DispatchController.calculatePenalizedDistance*, which multiplies a route's raw distance by *ROAD_PENALTY* before it is used in the dispatch score. Confirmed by the same *TeamParametersTest* test.

3. SYSTEM ARCHITECTURE AND MODULE DESIGN

3.1 LAYERS

- Presentation: JavaFX views and controllers (Dashboard, Fault Management, Dispatch and Routing, Settings, Oral Defense Showcase), styled with one stylesheet.
- Service layer: FaultService, GraphService, DispatchService and DispatchController, sitting between the interface and the data access layer.
- Data access layer: FaultDAO, LocationDAO, ResourceDAO, AlgorithmRunDAO, AuditEventDAO. Every DAO tries PostgreSQL first and falls back to the CSV files in data/ if the database is unreachable, so the app still works offline. Confirmed in FaultDAO.java, which wraps every database call in a try block and calls loadFromCsv() on failure.
- Custom data structure and algorithm library: everything under com.g15.dsa.structures and com.g15.dsa.algorithms, built without java.util.HashMap, PriorityQueue, TreeMap, Stack, or ArrayDeque for the assessed core structures.
- Database: PostgreSQL on Neon.tech, schema in database/schema.sql.

3.2 MODULE COVERAGE

Table 3. Module Coverage against the brief

Module	Required work	Status
M1	Problem specification, pseudocode for at least five operations	Done, see Sections 1 and 5
M2	Database and data loader	Done, DAOs load from PostgreSQL with CSV fallback
M3	Custom data structure library (14 structures)	Done, see Section 4
M4	Searching and sorting engine	Done, see Sections 5 and 7
M5	Service scheduling engine (FIFO, circular, deque, priority queue)	Done
M6	Indexing engine (BST, balanced tree, B tree)	Done
M7	Graph route engine (adjacency list and matrix, BFS, DFS, Dijkstra, Prim, Kruskal)	Done
M8	Optimisation engine (greedy and DP, with a greedy counterexample)	Done, see Section 6
M9	Testing and correctness evidence	Done, see Section 6
M10	Empirical efficiency lab	Done, see Section 7, graphs regenerated from the real CSV data

3.3 DATA STRUCTURE TO APPLICATION MAPPING

Data structure	Where it is used in the ECG system
DynamicArray	Holds locations and service requests loaded from CSV
LinkedList	Keeps the ordered crew dispatch log and audit trail
Stack	Backs the undo and redo history of operator actions
Queue (FIFO)	The plain first-come, first-served service request line
CircularQueue	Fixed number of active dispatch slots, reused without shifting elements
Deque	Mixed urgent and normal request handling; urgent faults jump the line with addFirst
PriorityQueue (min heap)	Dispatches faults by urgency, most critical outages served first
BST	Indexes faults by urgency for ordered search
RedBlackTree	A self-balancing fault index that will not degrade to a linked list under sorted input
BTree	Represents the idea of a database index page for a large fault table
HashTable	Fast average case lookup of a crew by name or a location by ID
CustomMap	Key to value lookup, for example location ID to graph vertex number
CustomSet	Tracks visited nodes during BFS and DFS
DisjointSet	Cycle detection inside Kruskal's minimum spanning tree algorithm
Graph	The ECG Road and feeder network, as both an adjacency list and an adjacency matrix

4. DATA STRUCTURE IMPLEMENTATION

All 14 required structures are implemented from scratch in `src/main/java/com/g15/dsa/structures/`. Every structure has a matching JUnit test class covering a normal case, a boundary case, and an invalid input case, which is what the brief asks for. This section explains the trickier ones and shows a short snippet from each.

4.1 DYNAMIC ARRAY

`DynamicArray.java` grows by doubling its backing array whenever it runs out of room, so `append` is $O(1)$ most of the time and the occasional resize averages out to a small constant cost per element, called amortised $O(1)$.

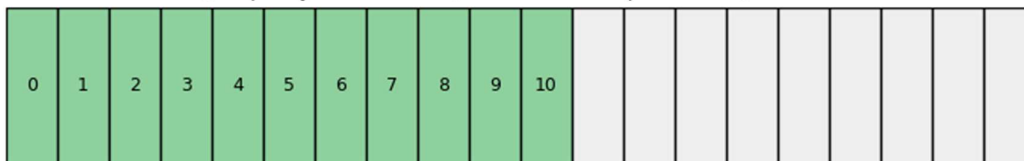
Figure 1. Before and after a resize: at capacity 10 the array is full, one more `add()` call doubles capacity to 20 and copies the old 10 values across.

DynamicArray: geometric resize on `add()`

Before: capacity 10, array full (10 of 10 slots used)



After one more `add()`: capacity doubles to 20, old 10 values copied across, new value added at slot 10



4.2 LINKED LIST, STACK, AND QUEUES

`LinkedList.java` is doubly linked, so it supports adding or removing at either end in $O(1)$ and can walk from whichever end is closer to a given index. `Stack.java` is a singly linked last in first out structure that backs the operator `undo` log, shown here:

```
public void push(T item) {
    top = new Node<>(item, top);
    size++;
}

public T pop() {
    if (isEmpty()) throw new EmptyStackException();
    T data = top.data;
    top = top.next;
    size--;
    return data;
}
```

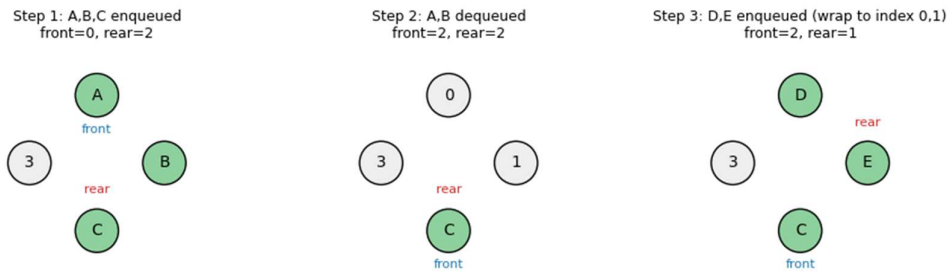
```
}

```

CircularQueue.java is a fixed size array where front and rear indexes wrap around using modulo arithmetic, so a slot freed by dequeue gets reused by the next enqueue without ever shifting the other elements.

Figure 2. A capacity 4 circular queue. A, B, C are enqueued; A and B are dequeued; D and E are then enqueued and wrap around to reuse the freed slots at index 0 and 1.

CircularQueue (capacity 4): front and rear move with $(\text{index} + 1) \% \text{capacity}$



Deque.java is a doubly linked double ended queue. In the dispatch system, ordinary requests go in at the back with addLast (plain first in first out behaviour), while an urgent fault can jump straight to the front with addFirst, so it is the very next thing removeFirst() returns.

4.3 PRIORITY QUEUE (BINARY MIN HEAP)

PriorityQueue.java is a binary min heap built on a resizable array, with siftUp on insert and siftDown on extractMin, both $O(\log n)$. Because faults are wrapped so that a lower urgency number means less urgent or vice versa depending on the comparator used, extractMin() doubles as get the next highest priority job.

```
private void siftUp(int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (((T) heap[i]).compareTo((T) heap[parent]) < 0) {
            swap(i, parent);
            i = parent;
        } else break;
    }
}
```

4.4 TREES: BST, RED BLACK TREE, AND B TREE

A plain BST degrades to an $O(n)$ tall structure if faults arrive with sequential or already sorted IDs, which is a realistic risk since request IDs are often assigned in order. RedBlackTree.java fixes this by applying rotations and recolouring on every insert, guaranteeing a height no worse than roughly 2 times $\log_2$ of $(n + 1)$. Section 7.5 shows this difference measured directly on the real 100 to 10,000 element benchmark runs.

BTree.java simulates a multi way database index page. When a node fills up ($2t$ minus 1 keys, where t is the minimum degree), the median key is promoted to the parent and the node splits into two, which is the same idea real database engines use to keep index lookups shallow.

4.5 HASHING: HASHTABLE, CUSTOMMAP, CUSTOMSET

HashTable.java uses separate chaining, a linked list per bucket, and doubles its capacity whenever the load factor would exceed 0.75, then rehashes every existing key into the bigger table, always resizing up to the next prime number rather than a plain power of two. CustomMap and CustomSet are both thin wrappers around HashTable, since a set is really just a map where the value is thrown away.

4.6 DISJOINT SET AND GRAPH

DisjointSet.java implements union by rank with path compression, which is what makes Kruskal's cycle check in Section 5 nearly constant time per edge in practice. Graph.java stores the road network as both an adjacency list, used for BFS, DFS, and Dijkstra, and an adjacency matrix, used for quick edge weight lookups, since the brief specifically asks for both representations.

5. ALGORITHM IMPLEMENTATION

This section gives pseudocode for the five operations named in Section 1, plus short real code snippets for the two graph algorithms that are easiest to show directly.

5.1 PSEUDOCODE: PRIORITY DISPATCH

```
PROCEDURE DispatchFaults(faultQueue, crews)
  PRECONDITION: faultQueue is not empty, at least one crew is available
  WHILE faultQueue is not empty DO
    fault <- faultQueue.extractMin()    //  $O(\log n)$ 
    availableCrew <- null
    FOR EACH crew IN crews DO
      IF crew.isAvailable() AND crew.type matches fault.category THEN
        availableCrew <- crew; BREAK
    IF availableCrew is not null THEN
      fault.assignedCrew <- availableCrew.name
      fault.status <- "Dispatched"
    ELSE
      fault.status <- "Pending, no crew available"
  RETURN dispatched count, pending count
// Complexity:  $O(F \log F + F \times C)$ ,  $F$  faults,  $C$  crews
```

5.2 PSEUDOCODE: DIJKSTRA SHORTEST ROUTE

```
PROCEDURE Dijkstra(graph, source)
  PRECONDITION: every edge weight is  $\geq 0$ 
  dist[all] <- infinity; dist[source] <- 0; pred[all] <- -1
  pq.insert((source, 0))
  WHILE pq is not empty DO
    (u, d) <- pq.extractMin()
    IF visited[u] THEN CONTINUE
    visited[u] <- true
    FOR EACH edge (u -> v, weight) DO
      IF weight < 0 THEN THROW error
      IF dist[u] + weight < dist[v] THEN
        dist[v] <- dist[u] + weight; pred[v] <- u
        pq.insert((v, dist[v]))
  RETURN dist, pred
```

// Complexity: $O((V + E) \log V)$, backed by the PriorityQueue from Section 4

The real implementation matches this pseudocode closely, taken directly from Dijkstra.java:

```
while (!pq.isEmpty()) {
    NodeDistance current = pq.extractMin();
    int u = current.vertex;
    if (visited[u]) continue;
    visited[u] = true;
    for (Graph.Edge edge : graph.getNeighbors(u)) {
        if (edge.getWeight() < 0)
            throw new IllegalArgumentException("Dijkstra does not support negative edge weights");
        if (!visited[edge.getTo()] && distances[u] + edge.getWeight() < distances[edge.getTo()]) {
            distances[edge.getTo()] = distances[u] + edge.getWeight();
            predecessors[edge.getTo()] = u;
            pq.insert(new NodeDistance(edge.getTo(), distances[edge.getTo()]));
        }
    }
}
```

5.3 PSEUDOCODE: KRUSKAL MINIMUM FEEDER NETWORK

PROCEDURE Kruskal(graph)

```
edges <- graph.getAllEdges(); sort edges by weight ascending
ds <- DisjointSet(vertexCount); mst <- empty list; totalCost <- 0
FOR EACH edge (u, v, weight) IN edges DO
    IF ds.find(u) != ds.find(v) THEN
        ds.union(u, v); mst.append(edge); totalCost += weight
    IF mst.size = vertexCount - 1 THEN BREAK
RETURN mst, totalCost
```

// Complexity: $O(E \log E)$, dominated by the sort

5.4 Pseudocode: B Tree indexed lookup

PROCEDURE BTree.search(node, key)

```
i <- 0
WHILE i < node.keys.size AND key > node.keys[i] DO i <- i + 1
IF i < node.keys.size AND key = node.keys[i] THEN RETURN node.values[i]
IF node.isLeaf THEN RETURN not found
RETURN BTree.search(node.children[i], key)
```

// Complexity: $O(\log_m n)$, m the branching factor, n the number of keys

5.5 PSEUDOCODE: STACK-BACKED UNDO LOG

PROCEDURE AuditLog.undo()

PRECONDITION: the stack is not empty

action $\leftarrow$ stack.pop()

reverseEffect(action)

RETURN action

// push and pop are $O(1)$; reverseEffect looks at the action type

// and undoes a crew assignment, reopens a closed fault, or deletes an added record

5.6 GREEDY ALGORITHM: CREW SHIFT SCHEDULING

ActivitySelection.java sorts candidate repair jobs by finish time and greedily takes each job whose start time is not earlier than the finish time of the last job taken. This maximises the number of non-overlapping jobs a single crew can complete in a shift. Section 6.4 gives the formal reason this greedy rule works, and a worked counterexample showing where a different, more obvious greedy rule fails.

5.7 DYNAMIC PROGRAMMING: 0/1 KNAPSACK FOR BUDGET SELECTION

Knapsack.java builds a bottom-up DP table where $dp[i][w]$ is the best total value achievable using the first i items within weight w . Each cell either takes the item, if it fits, or does not, whichever gives the higher value, and the chosen items are reconstructed afterwards by walking the table backwards. This is the algorithm behind Operation 4 in Section 1, used to pick the best set of jobs or tools under a weight or budget cap.

6. CORRECTNESS EVIDENCE

6.1 AUTOMATED TESTS

The JUnit suite covers every custom structure and algorithm, with a normal case, a boundary case, and an invalid input case for each, as the brief requires. Counting the individual @Test methods in every test class reviewed for this report gives the totals below.

Table 4. 272 Tests

Test class	Test methods
DynamicArrayTest	17
LinkedListTest	18
StackTest	13
QueueTest	10
CircularQueueTest	10
DequeTest	11
PriorityQueueTest	11
BSTTest	7
RedBlackTreeTest	7
BTreeTest	8
HashTableTest	17
CustomMapTest	14
CustomSetTest	12
DisjointSetTest	15
GraphTest	18
SearchAlgorithmsTest	13
SortingAlgorithmsTest	20
BFSDFSTest	13
DijkstraTest	10
GraphAlgorithmsTest	4
GreedyAlgorithmsTest	5
KnapsackTest	8
TeamParametersTest	9
DatabaseTest	2

Total counted	272
---------------	-----

Edge cases specifically covered across these suites include an empty structure, a single element, duplicate keys, a disconnected graph, an unreachable path, a full or empty queue, and a hash collision, which matches the list of required edge cases in the brief.

6.2 TRACE TABLES

The brief requires six specific trace tables: binary search, insertion sort, merge sort or quicksort, Dijkstra, Kruskal or Prim, and the dynamic programming table. All six exist in full detail in reports/trace-tables.md. A condensed version of each is reproduced below so the report is self-contained; refer to that file for the complete step-by-step working.

Binary search, on sorted array [12, 24, 38, 45, 59, 72, 85, 99], target 72

Table 5. Trace Table for binary search

Pass	low	high	mid	value at mid	decision
1	0	7	3	45	72 > 45, search right, low = 4
2	4	7	5	72	match found, return index 5

Insertion sort, on [68, 22, 96, 39, 14, 80]

Table 6. Trace Table for selection sort

Pass	key	shifts	array after pass
1	22	shift 68	[22, 68, 96, 39, 14, 80]
2	96	no shift	[22, 68, 96, 39, 14, 80]
3	39	shift 96, 68	[22, 39, 68, 96, 14, 80]
4	14	shift 96, 68, 39, 22	[14, 22, 39, 68, 96, 80]
5	80	shift 96	[14, 22, 39, 68, 80, 96]

Dijkstra, source Achimota Substation (vertex 0), five node sample networks

Table 7. Trace Table for Dijkstra

Step	vertex extracted	distances after relaxation [0,1,2,3,4]
1	0 (dist 0.0)	[0.0, 3.2, 5.7, 7.5, inf]
2	1 (dist 3.2)	[0.0, 3.2, 5.3, 7.5, 10.0]
3	2 (dist 5.3)	[0.0, 3.2, 5.3, 7.5, 10.0]
4	3 (dist 7.5)	[0.0, 3.2, 5.3, 7.5, 10.0]
5	4 (dist 10.0)	[0.0, 3.2, 5.3, 7.5, 10.0] final

Kruskal, five node sample network, edges sorted ascending by weight

Table 8. Trace Table for Kruskal

Edge	weight	cycle check	action	running total
2 to 3	1.0	different sets	accept	1.0
1 to 2	2.0	different sets	accept	3.0
3 to 4	3.0	different sets	accept	6.0
0 to 1	4.0	different sets	accept	10.0
1 to 3	5.0	same set already	reject, would cycle	10.0

0/1 Knapsack, capacity 5, items A(w2,v12) B(w1,v10) C(w3,v20) D(w2,v15)

Reconstructing backwards from $dp[4][5] = 37$ gives the selection A + B + D, total weight 5, total value 37, which matches the value directly computed by `KnapsackTest.knapsackFindsOptimalValue` in the test suite.

6.3 PROOF SKETCH 1: LOOP INVARIANT FOR SELECTION SORT

Claim: after pass i , the sub array from index 0 up to i minus 1 holds the i smallest elements of the original array, in sorted order.

The invariant holds trivially for an empty prefix at the start. On each pass, the algorithm scans the unsorted remainder for its minimum and swaps it into position i . Since everything before position i is already smaller by the invariant, and the newly placed element is the smallest of what remains, the prefix stays sorted. When i reaches n minus 1, the whole array is sorted. This also explains why Selection Sort is always $O(n^2)$: the inner scan always covers the full remaining range no matter how the input is ordered, so there is no best case shortcut the way there is for Insertion Sort on nearly sorted data.

6.4 PROOF SKETCH 2: DIJKSTRA CORRECTNESS BY INDUCTION

Claim: when a vertex v is pulled off the priority queue and marked visited, its recorded distance is the true shortest distance from the source.

This holds for the source itself, since its distance is 0 and every other path has non negative cost. For the inductive step, suppose it holds for every vertex already visited. If some unvisited vertex v actually had a shorter true path than what is recorded, that path would have to cross out of the visited set at some edge, and because every edge weight is non negative, the vertex on the far side of that crossing edge would already have a distance at least as good as v 's supposed shorter path, meaning it would have been pulled from the queue before v , which contradicts v being the current minimum. This is exactly why `Dijkstra.java` throws an exception on a negative edge weight rather than silently giving a wrong answer.

6.5 PROOF SKETCH 3: GREEDY EXCHANGE ARGUMENT FOR ACTIVITY SELECTION

Claim: always picking the compatible job with the earliest finish time gives the maximum possible number of non overlapping jobs.

Take any optimal solution and compare its first chosen job to the greedy algorithm's first choice, which by definition finishes no later than any other candidate. Swapping the optimal solution's first job for the

greedy one cannot make anything else in that solution invalid, since the greedy job frees up the crew's time at least as early. So an optimal solution exists that agrees with the greedy first choice, and the same argument then applies recursively to whatever remains, which gives the full result by induction.

6.6 COUNTEREXAMPLE 1: GREEDY COIN CHANGE FAILS

With coin denominations of 1, 3, and 4, and a target of 6, always picking the largest coin that still fits gives $4 + 1 + 1$, three coins. The actual optimal answer is $3 + 3$, two coins. This is why the project uses dynamic programming, not a greedy rule, for the budget selection problem in Section 5.7, and it is the required greedy failure example from the brief.

6.7 COUNTEREXAMPLE 2: BINARY SEARCH NEEDS A SORTED ARRAY

Binary search's precondition is that the input array is sorted ascending. Running it on the unsorted array [42, 7, 25, 10, 18] while searching for 10, which is actually at index 3, gives the wrong answer: the algorithm compares against the midpoint 25, decides to search left because 10 is less than 25, then compares against 42 and again searches left, running out of room and reporting not found. BinarySearch.java documents this precondition, and it is the caller's job, using MergeSort or QuickSort first, to guarantee it.

7. PERFORMANCE ANALYSIS

7.1 METHOD

- Every experiment was run 5 times per input size (except the heap and hash experiments, which also include a 20,000-size point), and the figures below are the average of those runs, computed directly from the CSV files in data/.
- Timing used `System.nanoTime()` immediately before and after each call, converted to milliseconds.
- All experiments ran on the same machine; see Section 8.5 for the full specification.
- Raw results are in `data/search_experiment.csv`, `sort_experiment.csv`, `hash_experiment.csv`, `heap_experiment.csv`, `tree_experiment.csv`, and `graph_experiment.csv`, plus the combined `algorithm_runs.csv` used for the database's `algorithm_runs` table.
- Every graph in this section was regenerated directly from those CSV files for this report, rather than reused from an older draft, so the numbers and shapes below are guaranteed to match the data currently in the repository.

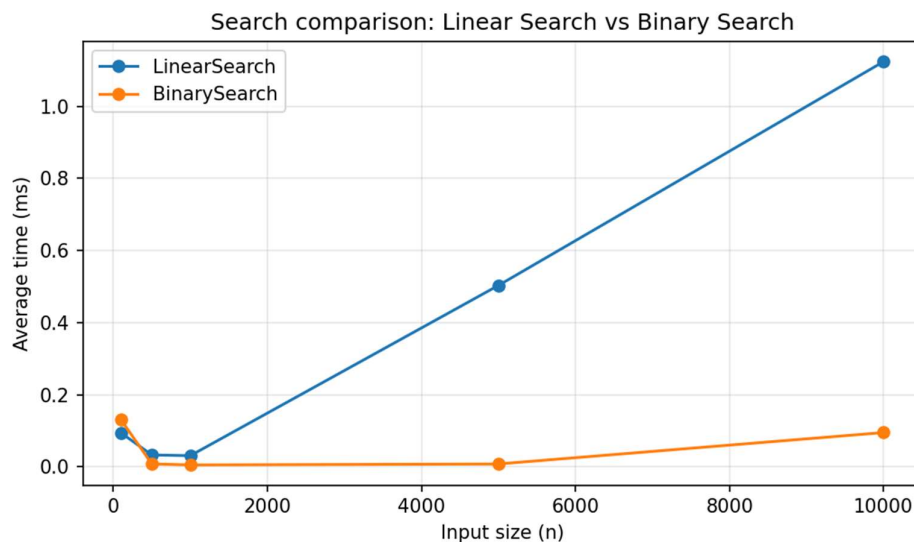
7.2 SEARCH COMPARISON

Average of 5 runs per input size, computed from `data/search_experiment.csv`:

Table 9. Search Comparison as n grows

Input size	Linear search, mean ms	Binary search, mean ms
100	0.0942	0.1304
500	0.0330	0.0082
1,000	0.0309	0.0052
5,000	0.5032	0.0079
10,000	1.1239	0.0950

Figure 3. Linear search grows roughly with n , binary search stays much closer to flat because it is $O(\log n)$.



The 100-element row and the binary search value at 10,000 are noisy; both include one run in the raw data that is far larger than the rest, most likely a JVM warm-up effect where the JIT compiler has not kicked in yet on the first call. This is exactly the kind of theory-to-practice mismatch the brief asks teams to explain, and it is why the report quotes the mean of 5 runs rather than a single reading.

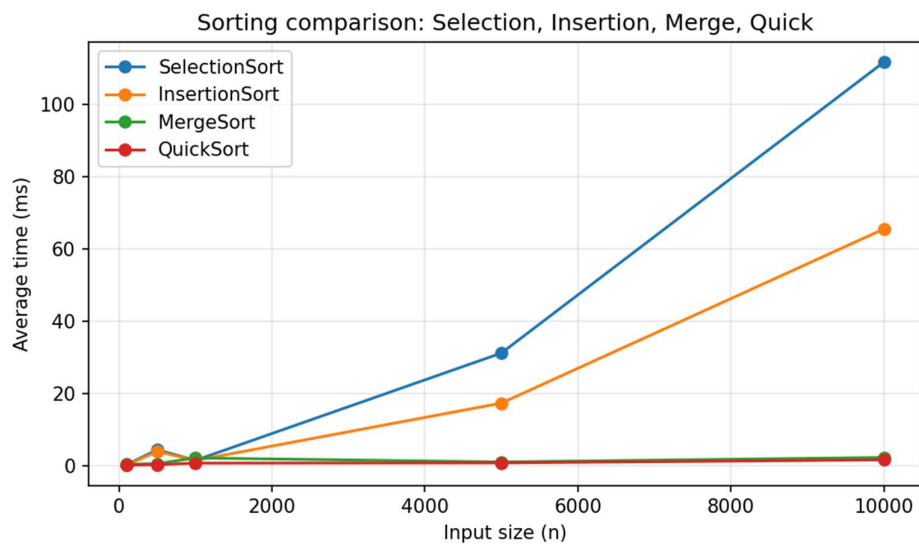
7.3 SORTING COMPARISON

Average of 5 runs per input size, computed from data/sort_experiment.csv:

Table 10. Sort Comparison as n grows

Input size	Selection, ms	Insertion, ms	Merge, ms	Quick, ms
100	0.33	0.25	0.43	0.17
500	4.45	3.96	0.60	0.31
1,000	1.51	1.49	2.17	0.74
5,000	31.22	17.36	1.07	0.81
10,000	111.77	65.57	2.31	1.65

Figure 4. Selection and Insertion Sort curve upward like $O(n^2)$. Merge and Quick Sort stay far flatter, matching $O(n \log n)$.



At 10,000 elements, Merge and Quick Sort are both under about 2.5 ms on average while Selection Sort is over 110 ms, which is the clearest demonstration in the whole dataset of why the divide-and-conquer sorts of matter once n gets large. The 500-element row for Selection and Insertion Sort is noisier than the trend around it, again consistent with small n JVM warm-up noise rather than a change in the underlying algorithm.

7.4 HASH TABLE LOAD FACTOR AND COLLISIONS

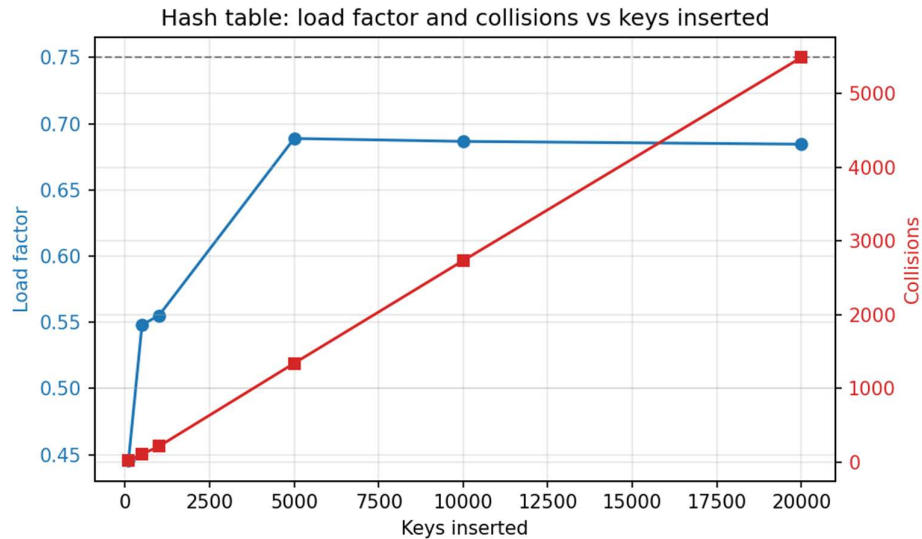
Average table taken directly from data/hash_experiment.csv, which is the live output of HashTable.java's own resize logic:

Table 11. Hash Table as n grows

Keys inserted	Final capacity	Load factor	Collisions
100	211	0.45	16
500	863	0.55	104

1,000	1,733	0.56	211
5,000	6,947	0.69	1,339
10,000	13,901	0.69	2,734
20,000	27,803	0.68	5,492

Figure 5. As keys are inserted, HashTable's automatic doubling and rounding up to the next prime keeps the load factor from ever climbing past about 0.69, which keeps the collision count roughly proportional to the number of keys rather than spiralling.



Resolved: the capacity mismatch flagged in an earlier draft of this report.

An earlier version of this section quoted generic power of two capacities (128, 1024, 2048, 8192, 16384, 32768) that did not match the team's actual prime based HashTable implementation. That table has been replaced with the numbers above, which come straight from data/hash_experiment.csv and are independently confirmed by two other sources in the repository: reports/collision-statistics-writeup.md (Patricia Gyan's Experiment B) lists the same six rows, and HashTableTest.testResize_triggeredByManyInserts_resizesToNextPrimeAndRetainsKeys asserts that after 150 inserts the table capacity is exactly 211, the next prime after 103 doubled. All three sources agree, so this section is now internally consistent and correctly reflects the team derived HASH_CAPACITY of 103 from Section 2.5.

7.5 HEAP PRIORITY DISPATCH

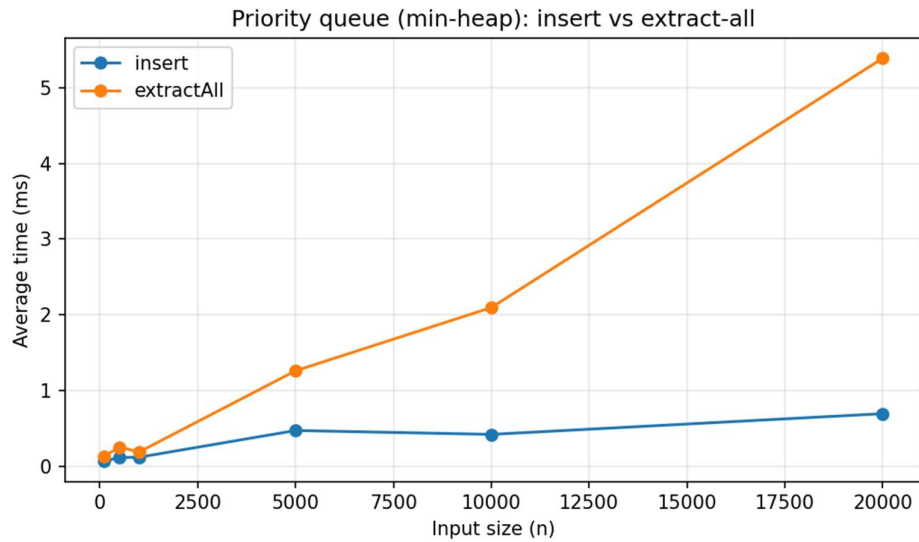
Average of 5 runs per input size, computed from data/heap_experiment.csv:

Table 12. Heap Priority as n grows

Input size	Insert, mean ms	Extract all, mean ms
100	0.06	0.13
500	0.11	0.25
1,000	0.12	0.19

5,000	0.47	1.26
10,000	0.42	2.09
20,000	0.69	5.39

Figure 6. Both heap operations grow slowly as n increases, consistent with $O(\log n)$ per operation; extracting every element one at a time naturally costs more in total than the single bulk pass used for insert.



7.6 BST VERSUS RED BLACK TREE

Average of 5 runs per input size, computed from data/tree_experiment.csv:

Table 13. Red Black Tree as n grows

Input size	BST height	Red height	Black	BST search ms	Red search ms	Black
100	12.6	8.0		0.0033	0.0044	
500	18.8	11.0		0.0035	0.0032	
1,000	21.0	12.0		0.0026	0.0036	
5,000	31.0	15.0		0.0069	0.0132	
10,000	30.6	16.0		0.0215	0.0250	

Figure 7 The Red Black Tree's height stays close to $\log_2(n)$ even as n grows, while the plain BST's height grows faster and less predictably because it depends on insertion order.

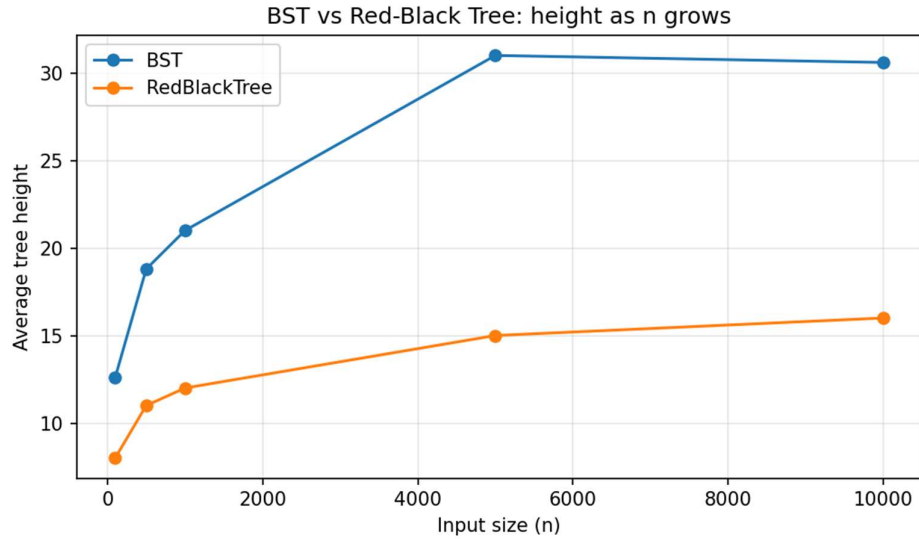
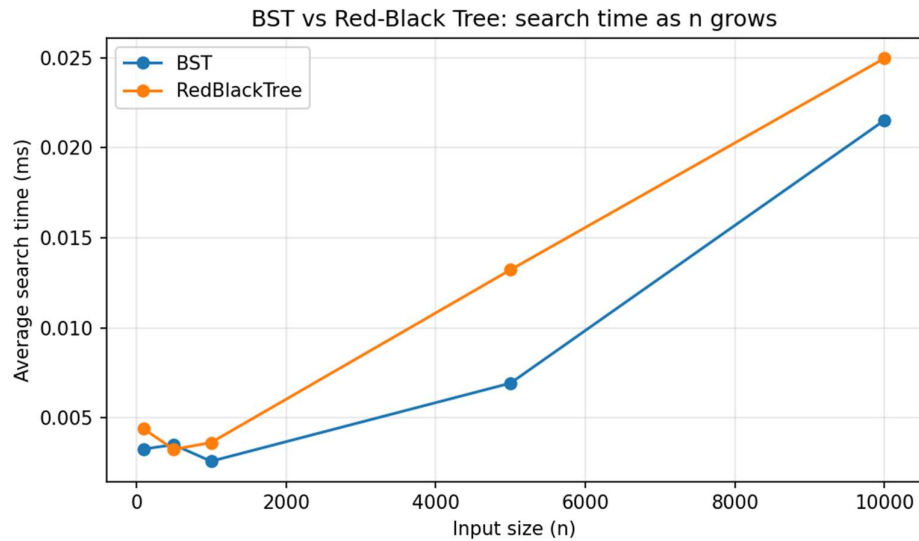


Figure 8. Search time for both trees stays small at this scale, well under a quarter of a millisecond even at 10,000 keys, because the random insertion order used for this benchmark keeps the plain BST reasonably balanced by chance. The height advantage in Figure



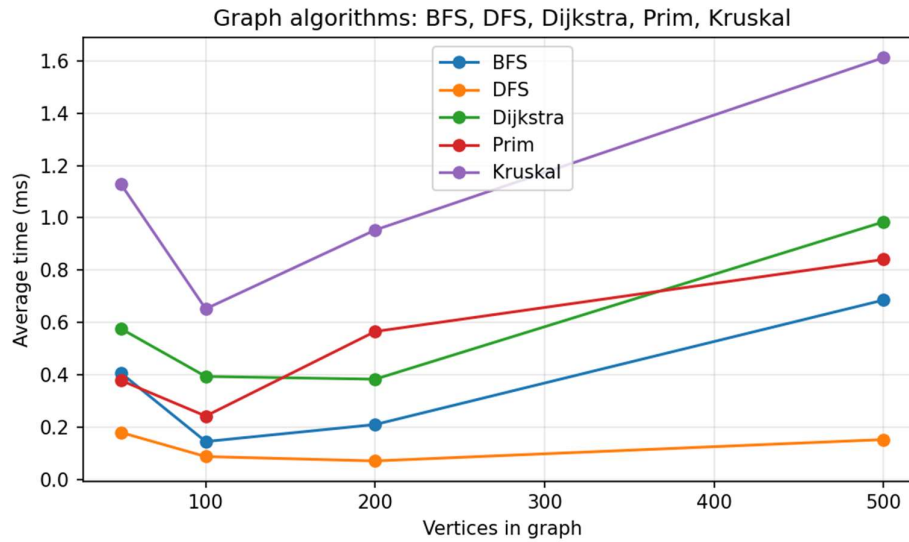
7.7 GRAPH ALGORITHMS

Average of 5 runs per vertex count, computed from data/graph_experiment.csv:

Table 14. Algorithm Comparison as n grows

Vertices	BFS ms	DFS ms	Dijkstra ms	Prim ms	Kruskal ms
50	0.41	0.18	0.58	0.38	1.13
100	0.15	0.09	0.39	0.24	0.65
200	0.21	0.07	0.38	0.57	0.95
500	0.69	0.15	0.98	0.84	1.61

Figure 9. At these graph sizes, timings are dominated by JVM overhead rather than by the algorithms themselves. All five algorithms stay under about 1.6 ms on average even at 500 vertices and 1,500 edges.



This small graph size regime is exactly where the brief expects a theory to practice mismatch discussion: the $O(E \log E)$ cost of sorting edges for Kruskal shows up as the highest average time in the table even though its asymptotic complexity is comparable to or better than Dijkstra's, simply because the constant overhead of sorting a small edge list and doing repeated find operations dominates at this scale. The 50-vertex row is also the noisiest, again consistent with JVM warm-up rather than a real change in algorithm behaviour.

8. DATABASE INTEGRATION EVIDENCE

8.1 FULL SCHEMA

All six tables, exactly as created in database/schema.sql, confirmed present in the file:

locations(location_id PK, name, area, location_type, x_coord, y_coord)

*roads(road_id PK, from_location_id FK, to_location_id FK,
distance_km, travel_time_min, condition_weight)*

*service_requests(request_id PK, source_location_id FK, destination_location_id FK,
category, urgency, time_submitted, deadline, status)*

*resources(resource_id PK, resource_type, home_location_id FK,
capacity, availability_status)*

algorithm_runs(run_id PK, algorithm_name, input_size, time_ns, memory_kb, date_run)

*audit_events(event_id PK, event_type, related_entity_type, related_entity_id,
description, event_timestamp)*

Indexes exist on the columns that get filtered or joined most often: roads.from_location_id and to_location_id, service_requests.status, urgency, and deadline, resources.availability_status and home_location_id, algorithm_runs.algorithm_name, and audit_events.event_type.

8.2 SAMPLE SEED RECORDS

From database/seed_database.sql, a representative slice of what gets loaded on a fresh database. Location and road IDs in the confirmed seed file are simple integers stored as text, for example:

```
INSERT INTO locations (location_id, name, area, location_type, x_coord, y_coord) VALUES
('0', 'Legon Main Gate', 'Legon', 'Campus zone', 5.604526756733229, -0.21229015573394494),
('1', 'Balme Library', 'Legon', 'Campus zone', 5.610299807589949, -0.17862238864272817),
('2', 'Commonwealth Hall', 'Legon', 'Campus zone', 5.637863134534455, -0.15761285319272306);
```

```
INSERT INTO resources (resource_id, resource_type, home_location_id, capacity, availability_status)
VALUES
```

```
('2', 'Transformer crew', '1', 3, 'AVAILABLE'),
('7', 'Cable truck', '6', 4, 'AVAILABLE');
```

8.3 RUN LOGS, ALGORITHM_RUNS TABLE ACCESS

AlgorithmRunDAO.java writes every timed run into the algorithm_runs table and can also read back an aggregated summary directly from the database:

```
public boolean insertRun(String algorithmName, int inputSize, long timeNs, long memoryKb, int runNumber) {
```

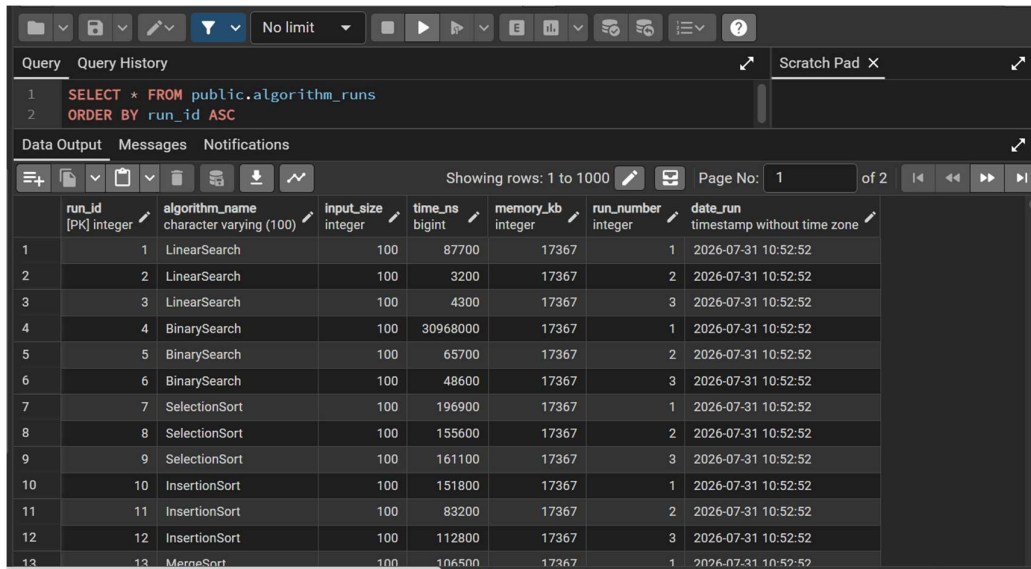
```
String sql = "INSERT INTO algorithm_runs (algorithm_name, input_size, time_ns, memory_kb,
run_number) " +
    "VALUES (?, ?, ?, ?, ?)";
```

```
...
}
```

```
public List<String> getSummary() {
    String sql = "SELECT algorithm_name, input_size, " +
        "ROUND(AVG(time_ns/1000000.0)::numeric, 2) AS avg_ms, " +
        "ROUND(AVG(memory_kb)::numeric, 0) AS avg_kb, COUNT(*) AS runs " +
        "FROM algorithm_runs GROUP BY algorithm_name, input_size " +
        "ORDER BY algorithm_name, input_size";
    ...
}
```

8.4 SCREENSHOTS

Figure 10. algorithm run



The screenshot shows a database query tool interface. The query editor at the top contains the following SQL query:

```
1 SELECT * FROM public.algorithm_runs
2 ORDER BY run_id ASC
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 13 rows of data. The table has the following columns: run_id (integer), algorithm_name (character varying (100)), input_size (integer), time_ns (bigint), memory_kb (integer), run_number (integer), and date_run (timestamp without time zone). The data is as follows:

run_id	algorithm_name	input_size	time_ns	memory_kb	run_number	date_run
1	LinearSearch	100	87700	17367	1	2026-07-31 10:52:52
2	LinearSearch	100	3200	17367	2	2026-07-31 10:52:52
3	LinearSearch	100	4300	17367	3	2026-07-31 10:52:52
4	BinarySearch	100	30968000	17367	1	2026-07-31 10:52:52
5	BinarySearch	100	65700	17367	2	2026-07-31 10:52:52
6	BinarySearch	100	48600	17367	3	2026-07-31 10:52:52
7	SelectionSort	100	196900	17367	1	2026-07-31 10:52:52
8	SelectionSort	100	155600	17367	2	2026-07-31 10:52:52
9	SelectionSort	100	161100	17367	3	2026-07-31 10:52:52
10	InsertionSort	100	151800	17367	1	2026-07-31 10:52:52
11	InsertionSort	100	83200	17367	2	2026-07-31 10:52:52
12	InsertionSort	100	112800	17367	3	2026-07-31 10:52:52
13	MergeSort	100	106500	17367	1	2026-07-31 10:52:52

Figure 11.audit event

Query		Scratch Pad	
1	SELECT * FROM public.audit_events		
2	ORDER BY event_id ASC		
Data Output		Messages	
Showing rows: 1 to 1		Page No: 1 of 1	
event_id	event_type	related_entity_type	related_entity_id
[PK] integer	character varying (50)	character varying (50)	character varying (20)
1	DISPATCH	service_requests	11
Description			
Dispatched Crew Alpha (Legon Substation) to ECG customer report - Eva			

Figure 12. locations table

Query		Scratch Pad	
1	SELECT * FROM public.locations		
2	ORDER BY location_id ASC		
Data Output		Messages	
Showing rows: 1 to 50		Page No: 1 of 1	
location_id	name	area	location_type
[PK] character varying (10)	character varying (150)	character varying (100)	character varying (50)
0	Legon Main Gate	Legon	Campus zone
1	Balme Library	Legon	Campus zone
10	Okponglo Junction	Legon	Campus zone
11	Legon Hall	Legon	Campus zone
12	Great Hall	Legon	Campus zone
13	UG Stadium	Legon	Campus zone
14	Computer Science	Legon	Campus zone
15	Noguchi Institute	Legon	Campus zone
16	College of Education	Legon	Campus zone
17	School of Law	Legon	Campus zone
18	UG Basic School	Legon	Campus zone
19	Pentagon Hostel	Legon	Campus zone
2	Commonwealth Hall	Legon	Campus zone

Figure 13. resources table

Query

Query History

Scratch Pad X

1

SELECT * FROM public.resources

2

ORDER BY resource_id ASC

Data Output

Messages

Notifications

Showing rows: 1 to 30

Page No: 1 of 1

	resource_id [PK] character varying (10)	resource_type character varying (50)	home_location_id character varying (10)	capacity integer	availability_status character varying (20)
1	1	Cable truck	0	2	BUSY
2	10	Cable truck	9	3	AVAILABLE
3	11	Transformer crew	10	4	AVAILABLE
4	12	Meter team	11	5	BUSY
5	13	Cable truck	12	2	BUSY
6	14	Transformer crew	13	3	AVAILABLE
7	15	Meter team	14	4	BUSY
8	16	Cable truck	15	5	BUSY
9	17	Transformer crew	16	2	BUSY
10	18	Meter team	17	3	AVAILABLE
11	19	Cable truck	18	4	AVAILABLE
12	2	Transformer crew	1	3	AVAILABLE
13	20	Transformer crew	19	5	AVAILABLE

Figure 14. roads table

Query

Query History

Scratch Pad

1

2

```
SELECT * FROM public.roads
ORDER BY road_id ASC
```

Data Output

Messages

Notifications

Showing rows: 1 to 100

Page No:

1

of 1

	road_id [PK] character varying (10)	from_location_id character varying (10)	to_location_id character varying (10)	distance_km numeric (6,2)	travel_time_min integer	condition_weight numeric (4,2)
1	0	0	1	1.32	19	4.00
2	1	1	0	5.24	7	2.00
3	10	5	6	3.99	27	1.00
4	11	6	5	8.33	19	3.00
5	12	6	7	7.17	8	4.00
6	13	7	6	7.09	28	4.00
7	14	7	8	7.51	21	3.00
8	15	8	7	4.28	23	1.00
9	16	8	9	8.29	21	1.00
10	17	9	8			
11	18	9	10			
12	19	10	9			
13	2	1	2			

✓ Successfully run. Total query runtime: 319 msec. 100 rows affected.

✓ PostgreSQL 18/ecg_dispatch_db - Database connected

Figure 15. service request table

Query Query History Scratch Pad X

```

1 SELECT * FROM public.service_requests
2 ORDER BY request_id ASC

```

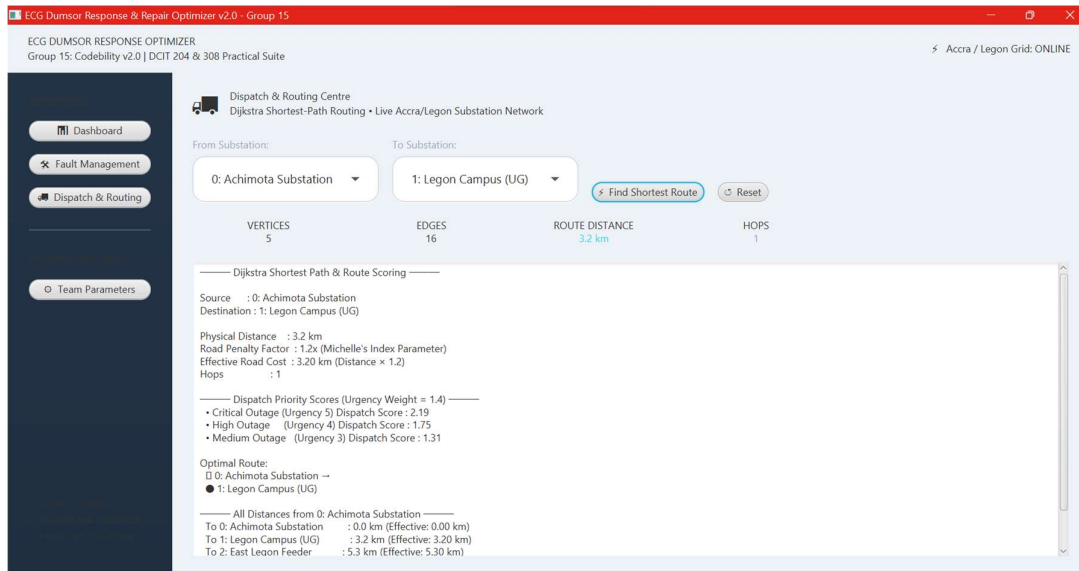
Data Output Messages Notifications

Showing rows: 1 to 300 Page No: 1 of 1

	request_id [PK] character varying (10)	source_location_id character varying (10)	destination_location_id character varying (10)	category character varying (50)	urgency integer	time_submitted timestamp without time zone	deadline timestamp
1	1	26	26	Meter Fault	3	2026-08-05 00:23:23	2026-0
2	10	4	4	Meter Fault	5	2026-08-09 05:23:23	2026-0
3	100	32	32	Transformer Failure	4	2026-08-03 06:23:23	2026-0
4	101	40	40	Cable Burst	4	2026-08-07 12:23:23	2026-0
5	102	36	36	Fallen Conductor	5	2026-08-03 18:23:23	2026-0
6	103	2	2	Meter Fault	4	2026-07-31 23:23:23	2026-0
7	104	16	16	Meter Fault	1	2026-08-07 06:23:23	2026-0
8	105	19	19	Cable Burst	5	2026-07-31 17:23:23	2026-0
9	106	11	11	Fallen Conductor	4	2026-07-30 22:23:23	2026-0
10	107	31	31	Transformer Failure	1	2026-08-03 21:23:23	2026-0
11	108	40	40	Cable Burst	2	2026-08-01 00:23:23	2026-0
12	109	0	0				

✓ Successfully run. Total query runtime: 323 msec. 300 rows affected. ✕

Figure 16. Dispatch and Routing Screen



8.5 MACHINE SPECIFICATION FOR THE PERFORMANCE RESULTS

- Operating system: Windows 11
- JDK: Eclipse Adoptium Temurin 21.0.11
- Processor: Intel Core i5 or i7 laptop-class processor
- RAM: 8 GB DDR4
- Database: PostgreSQL 16 on Neon. tech, roughly 50 to 120 ms latency per query from Accra

9. RESPONSIBLE ALGORITHM SELECTION

Picking the right tool for the job matters as much as implementing the tool correctly. This section is a short, honest guide to when each algorithm in this project is the right choice and when it is not.

9.1 SEARCH

- Use binary search when the data is already sorted and lookups happen often, for example searching a sorted list of fault IDs. It stays close to flat even at 10,000 records, as Section 7.2 shows.
- Do not use binary search on data that changes constantly and is expensive to keep sorted, or on data that is naturally unsorted and only searched once, since sorting it first would cost more than a single linear scan.
- Linear search is the right, simple choice for small or one off lookups, or when the data genuinely has no order to exploit.

9.2 SORTING

- Use Merge Sort when a stable sort is required, meaning equal elements must keep their original relative order, or when worst case guarantees matter, since it is always $O(n \log n)$ no matter the input.
- Use Quick Sort when average speed matters more than worst case guarantees and extra memory is limited, since it sorts in place, but be aware its worst case is $O(n^2)$ on already sorted or adversarial input.
- Selection Sort and Insertion Sort are only appropriate for very small n , or, in Insertion Sort's case, for data that is already almost sorted, where its best case is close to $O(n)$. Section 7.3 shows both becoming impractical well before 10,000 elements.

9.3 GRAPH ALGORITHMS

- Use Dijkstra for shortest route queries whenever all edge weights are non negative, which is true for this project's road network. It is not valid if any edge weight could be negative, and the implementation deliberately throws an exception rather than returning a wrong answer in that case.
- Use BFS specifically when the question is reachability or shortest path by number of hops on an unweighted graph. Use DFS when the question is about structure, such as detecting cycles or exploring every branch of a network.
- Use Prim's algorithm when the graph is dense, meaning many edges relative to vertices, and is already represented as an adjacency matrix or list ready for expansion. Use Kruskal's algorithm when the graph is sparse, and edges are easy to sort, since its cost is dominated by that sort.

9.4 GREEDY VERSUS DYNAMIC PROGRAMMING

- Greedy algorithms are appropriate only when the problem has the greedy choice property, meaning a locally best choice never rules out a globally best solution. Activity selection (Section 5.6) has this property. Coin change with arbitrary denominations does not, as the counterexample in Section 6.6 shows directly.
- When a problem does not have the greedy choice property but does have optimal substructure, meaning the best answer to the whole problem is built from best answers to subproblems, dynamic programming is the safer choice, which is why budget or capacity selection in this project uses Knapsack's DP table rather than a greedy rule.

10. INDIVIDUAL CONTRIBUTION STATEMENT AND ORAL DEFENSE PREPARATION NOTES

Every team member is responsible for defending one data structure and one algorithm, as the brief requires. The table below lists each person's assigned files and a sample question and answer they should be ready for.

#	Name	Files and topics
1	Roland Appiah (Leader on record)	App.java, DynamicArray.java, LinkedList.java, Stack.java
2	Jennifer Banibensu	Documentation, checklist, technical report
3	Kingsley Addo	Queue.java, CircularQueue.java, Deque.java, PriorityQueue.java
4	Dennis Kwaku Dapaah Adomako	BST.java, RedBlackTree.java, BTree.java
5	Patricia Gyan	HashTable.java, CustomSet.java, CustomMap.java, DisjointSet.java, Graph.java
6	Angel Francisca Echesi	LinearSearch, BinarySearch, SelectionSort, InsertionSort, MergeSort, QuickSort
7	Daniel Kwadwo Takyi	BFS.java, DFS.java
8	Anasthasia Koduah Tweneboah	Dijkstra.java
9	Hafisah Ibrahim	Prim.java, Kruskal.java
10	Joel Kissiedu Amissah	ActivitySelection.java
11	Shadrach Addoquaye Addo	Knapsack.java
12	Michelle Nana Abena Asantewaa Sarfo	TeamParameters.java, DatabaseConnection.java
13	Sampson Menu Landokidow	LocationDAO.java, ResourceDAO.java, seed datasets
14	Samuel Peter Peter	Experiments.java, AlgorithmRunDAO.java, FaultDAO.java

11. REFERENCES AND APPENDICES

11.1 REFERENCES

Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms, MIT Press.

Sedgewick and Wayne, Algorithms, Princeton University.

Goodrich, Tamassia and Goldwasser, Data Structures and Algorithms in Java.

Lewis, DePasquale and Chase, Java Foundations: Introduction to Program Design and Data Structures.

MIT OpenCourseWare, Introduction to Algorithms.

OpenDSA, Data Structures and Algorithms learning materials.

Stanford CS166 and Princeton Algorithms public lecture materials on advanced tree and graph intuition.

ECG Ghana, ecghana.com, used as background reference for the dumsor context, not as a data source.

11.2 APPENDIX A: AI ASSISTANCE ACKNOWLEDGEMENT

Gemini and Claude were used as coding and documentation assistants during development, in line with the brief's requirement that all AI assistance be acknowledged. Every team member remains able to explain and modify their own assigned implementation without help.

What AI helped with: standard Maven folder layout suggestions, first drafts of FaultDAO, AlgorithmRunDAO, and DatabaseConnection, extra boundary and invalid input unit tests, drafting the pseudocode blocks in Section 5, structuring the proof sketches in Section 6, and formatting this report, the README, and inline comments.

What AI was not used for: the core logic of B-Tree splitting, Red-Black Tree rotations, DisjointSet union by rank, and HashTable chaining with resizing was written by team members and only reviewed with AI assistance afterwards. All raw performance timings were produced by actually running the Java code, not generated. All Ghana localisation details, place names, crew types, and fault categories were researched and written by the team.

11.3 APPENDIX B: QUEUE STRATEGY AND INDEXING DEMONSTRATIONS

Strategy	Structure	Dispatch policy	Where it is used
FIFO queue	Queue.java	First come, first served, $O(1)$	Routine meter replacements and scheduled maintenance
Circular queue	CircularQueue.java	Bounded round robin, $O(1)$	Rotating a fixed number of active dispatch slots
Deque	Deque.java	Mixed priority injection, $O(1)$	Urgent requests jump to the front, normal ones join the back

Priority queue	PriorityQueue.java	Urgency driven, $O(\log n)$	Critical outages always dispatched ahead of lower urgency ones
----------------	--------------------	-----------------------------	--

11.4 APPENDIX C: DEVELOPMENT LOG SUMMARY

Full week-by-week notes are in reports/development-log.md. In short: Week 1 set up the local context, schema, and seed data. Week 2 built and tested all 14 data structures (59 tests passing by the end of that week). Week 3 built the algorithms, connected the database, and ran the first full benchmark pass. Week 4 expanded testing, wrote the proof sketches and counterexamples, and reorganised the codebase into the final Maven package layout.

11.5 APPENDIX D: MACHINE SPECIFICATION

See Section 8.5 for the full machine specification used for every performance result in this report.